

PmTracker.Web — Kompletní refaktor: analýza, plán a Codex prompty

ČÁST 1 — KONTEXT A PŘEHLED

Kontext

Aplikaci generoval AI (Codex) po částech — každá nová funkce nebo modul byl generován jako samostatný prompt bez kontextu toho, co již v projektu existovalo. Výsledkem je situace, kdy každý nový generovaný celek přinesl vlastní vrstvu abstrakce, vlastní konvence a vlastní sadu rozhraní — bez vědomí, že identická řešení již existují jinde v kódu. **Nic není nutně špatně napsané izolovaně. Výsledkem je však kombinace duplikátů, nekonzistencí a zbytečné hloubky delegace.**

Backend C# je technicky místy v pořádku — nullable types, sealed classes, správné EF Core vzory. Architektura jako celek však neodpovídá rozsahu aplikace: hierarchie tříd je obtížně sledovatelná a přidává vrstvu indirece bez přidané hodnoty.

Metriky

Metrika	Skutečnost	Co by mělo být
Složky/moduly	Services/ + Modules/ (duplikát)	Jen Services/
Soubory pro správu jednání	~35 souborů	1 service soubor
Soubory pro export (celkem)	46 souborů	1 service soubor
Vrstev delegace pro 1 metodu	10+	2 (Controller → Service → DB)
Závislostí v jedné třídě	20+ (SqlServerDataStore)	3-5
Řádků v site.js	9 016	~500 per modul
Async metod v celé aplikaci	0	100%
Views bez logiky	0	100%
UI konzistentních vzorů	různé na každé stránce	jednotný design systém

ČÁST 2 — KATALOG ZJISTĚNÍ

2.1 Architektura — dvě paralelní vrstvy pro totéž

`Services/Data/` a `Modules/` dělají přesně totéž pro stejné domény.

Skutečný call chain pro jednu DB operaci `BuildJednaniDetail(id)`:

```
JednaniController
  → IMeetingsQueries           [interface, Modules/]
  → MeetingsQueries           [1-liner: return handler.Handle(id)]
  → IBuildJednaniDetailQueryHandler [interface, Modules/]
  → BuildJednaniDetailQueryHandler [1-liner: return datastore.BuildJednaniDetail(id)]
  → IMeetingsDataStore         [interface, Modules/]
  → MeetingsDataStore          [1-liner: return datastore.BuildJednaniDetail(id)]
  → IPmTrackerDataStore        [interface, Services/Data/]
  → SqlServerDataStore         [deleguje na UseCase]
  → IMeetingDetailQueriesUseCase [interface, Services/Data/]
  → MeetingDetailQueriesUseCase [zde teprve pracuje]
```

10 tříd. Žádná z mezitříd nepřidává jedinou řádku logiky. Jen předávají volání dál.

Navíc `ProjektExistsQueryHandler` a `BuildProjektDetailQueryHandler` existují každý 3× v různých složkách s identickým kódem. DRY = 0.

Praktický dopad: při debugování bugy v `BuildJednaniDetail` vývojář prochází `JednaniController` → `MeetingsQueries` → handler → `DataStore` → `UseCase` — čtyři skoky přes čtyři soubory, než se dostane k řádce kde se bug skutečně nachází. Každý z těchto souborů je podezřelý, každý musí být přečten.

2.2 God class `SqlServerDataStore`

```
public sealed class SqlServerDataStore :
    IPmTrackerDataStore,           // rozhraní pro vše
    IProjectDetailComposition,     // rozhraní pro projekty
    IDictionariesQueriesComposition, // rozhraní pro číselníky
    IRecordEditorQueriesComposition, // rozhraní pro záznamy
    IRecordWriteCommandsComposition, // rozhraní pro zápis záznamů
    IDictionariesCommandsComposition // rozhraní pro zápis číselníků
```

Implementuje 6 rozhraní, injektuje 20+ závislostí v konstruktoru.

Jeden soubor zodpovídá za celou aplikaci. SRP = 0.

2.3 Celá aplikace je synchronní

```
// Každý z těchto volání blokuje ASP.NET thread:
var meeting = dbContext.Jednani.FirstOrDefault(x => x.Id == id);
var project = dbContext.Projekty.First(x => x.Id == meeting.ProjektId);
var attendance = dbContext.Ucast.Where(...).ToList();
```

Žádný `await`. Žádné `*Async` metody. Toto není hypotetický problém „až přijde zátěž“ — je to problém už dnes, při každém requestu.

ASP.NET Core má omezený thread pool. Synchronní DB volání nezablokuje jen request — zablokuje fyzický OS thread po celou dobu čekání na SQL Server. Při průměrné latenci DB dotazu 150–300ms: 20 souběžných requestů = 20 threadů čekajících nečinně na disk. IIS thread pool se vyčerpá, nové requesty čekají ve frontě, UI zamrzá. To se nestane při 2 uživateli. Stane se při 10, při deploy nové verze, při pomalém SQL Serveru. A až se to stane, v logu nebude žádná smysluplná chyba — jen timeouty a 503.

Async/await není optimalizace. Je to správné použití platformy, na které aplikace běží.

2.4 Načítání celých tabulek do paměti

```
// MeetingDetailQueriesUseCase.BuildMeetingTasks():
var people = dbContext.Osoby.AsNoTracking().ToDictionary(x => x.Id);
// ↑ Načte VŠECHNY osoby z DB bez WHERE filtru.
// 500 lidí v systému? Načte 500. Každý request.
```

Stejný problém s dalšími číselníkovými tabulkami — celá tabulka do paměti, pak filtr v C#.

Argument „v systému je jen 50 lidí“ mívá podstatu. Problém není škálování — je to **správnost**. Kód tvrdí, že potřebuje jednoho uživatele. Databáze mu pošle všechny. Tohle je chyba v logice, ne v kapacitě. Navíc: stačí jeden neplánovaný import dat (migrace ze starého systému, bulk insert) a „50 lidí“ se změní na 5 000 přes noc — bez jediné změny v kódu.

2.5 Over-fetching pro jednoduché operace

`JednaniController.TaskItemPartial` — chce zobrazit jeden úkol, zavolá `BuildJednaniDetail(id)`,

která načte **celý detail jednání**: všechny úkoly, všechny účastníky, všechny komentáře, stavy.

`JednaniController.SaveStatus` — chce jen ověřit `projektId` pro permission check. Zavolá `BuildJednaniDetail` — načte vše.

`JednaniController.AddMeetingParticipantModal` — totéž.

Jeden request na zobrazení jednoho úkolu = 6+ DB dotazů + vše do paměti.

2.6 N+1 DB operace v loopu

```
// SaveAttendance – 20 účastníků = 20 DB roundtripů:
foreach (var row in rows.Where(x => x.OsobaId > 0 && ...))
{
    _meetingsCommands.SaveAttendance(new SaveAttendanceCommand { ... }, CurrentUse
rContext);
    // každý průchod = 1 DB write + 1 SaveChanges + 1 transakce
}
```

Stačil by 1 batch write. Stejný problém v `SaveNotes`.

Každý průchod smyčkou otevírá transakci, zapíše, commituje, zavírá. 20 účastníků = 20 transakcí. Pokud DB selže na 15. záznamu, prvních 14 je commitováno, zbylých 5 ne — data jsou v nekonzistentním stavu bez jakékoli chybové zprávy uživateli. Jedna transakce kolem celé operace by to eliminovala úplně.

2.7 Service Locator anti-pattern v BaseController

```
// GetLocalNow():
var timeProvider = HttpContext.RequestServices.GetService<TimeProvider>();

// LogAjaxFailure():
var loggerFactory = HttpContext.RequestServices.GetService<ILoggerFactory>();
```

Service Locator je anti-pattern: závislosti vyřešené za runtime nejsou viditelné v signatuře třídy a patří do konstrukturu.

Konkrétní problém: závislost vyřešená přes `HttpContext.RequestServices` neexistuje v konstrukturu — není vidět v signatuře třídy. Vývojář čtoucí `BaseController` neví, že třída závisí na `TimeProvider` a `ILoggerFactory`, dokud neprojde celý kód metody po metodě. Při refaktoru DI registrací nebo při přidání nové DI scope (unit test, background job) tato závislost tiše selže s `NullReferenceException` za runtime, bez kompilační chyby.

2.8 Logika v Layout.cshtml

```
// _Layout.cshtml – layout by měl být HTML šablona, ne logický kód:
@{
    var currentController = ViewContext.RouteData.Values["controller"]?.ToString
().ToLowerInvariant();
```

```

        string NavClass(string controller) => currentController == controller ? "active" : "";
        var userContext = ViewBag.CurrentUserContext as CurrentUserContextViewModel;
        var canViewPeopleTab = userContext?.HasPermissionPrefix(PermissionKeys.PeoplePrefix) ?? false;
        var canViewCiselnikyTab = userContext?.HasPermissionPrefix(PermissionKeys.CiselnikyPrefix) ?? false;
        var canViewSettingsTab = userContext?.HasPermissionPrefix(PermissionKeys.SettingsPrefix) ?? false;
    }

```

Permission checky a výpočty přímo v layout souboru.

2.9 Logika ve Views místo ViewModels

Views/Projekty/Detail.cshtml — hned v prvním bloku:

```

@{
    var userContext = ViewBag.CurrentUserContext as CurrentUserContextViewModel;
    var canCreateMeetings = userContext?.HasPermission(PermissionKeys.MeetingsCreate, ...) ?? false;
    var canDeleteMeetings = userContext?.HasPermission(PermissionKeys.MeetingsEdit, ...) ?? false;
    var canManageTeam = userContext?.HasPermission(PermissionKeys.TeamManage, ...) ?? false;
    var canManageRecords = userContext?.HasPermission(PermissionKeys.RecordsEdit, ...) ?? false;
    var canManageSchedules = canManageRecords
        || (userContext?.HasPermission(PermissionKeys.RecordsScheduleEdit, ...) ?? false)
        || (userContext?.HasPermission(PermissionKeys.RecordsScheduleAdd, ...) ?? false);
    var createRecordEditorUrl = Url.Action(...);
    var returnToProjectUrl = Url.Action(...);
    var projectPrintUrl = Url.Action(...);
    var projectWordUrl = Url.Action(...);
    // ... dalších proměnných
}

```

Tento výpočet patří do ViewModelu — počítá se v Controlleru, View dostane hotové hodnoty.

2.10 UI nekonzistence — každá stránka vypadá jinak

Toto je jeden z největších problémů, který na první pohled nemusíš vidět v kódu, ale uživatel ho cítí každou vteřinou práce s aplikací.

Vzorový header na stránce "Osoby":

```
<section class="page-header">
  <h1>Osoby</h1>
  <div class="header-actions">...</div>
</section>
```

Vzorový header na stránce "Projekt Detail":

```
<section class="page-header page-header-compact">
  <div class="project-header-row">
    <a class="btn small ghost">Zpět</a>
    <span class="project-header-divider">|</span>
    <h1 class="project-title-inline">...</h1>
    <span class="pill">...</span>
    <span class="badge project-status-inline">...</span>
  </div>
</section>
```

Editor záznamu (`EditZaznamPage.cshtml`):

```
<section class="record-editor-page-shell">
  <div class="record-editor-page-header">
    <div class="record-editor-page-header-copy">
      <h1>...</h1>
    </div>
  </div>
  <section class="card record-editor-page-card">...</section>
</section>
```

Tři různé způsoby jak implementovat header stránky — každý s jinými CSS třídami, jinou HTML strukturou. Přidání čtvrté stránky (dashboard) přinese čtvrtý způsob. Vizuální výsledek: každá stránka má jiné zarovnání, jiné mezery, jiné umístění tlačítka "Zpět".

2.11 Middleware — buffering AJAX responses

```
// AjaxResponseContractGuardMiddleware:
await using var responseBuffer = new MemoryStream();
context.Response.Body = responseBuffer; // vše jde do RAM
```

```
await next(context); // celý request proběhne
// pak teprve validuje jestli je to JSON
```

Každý AJAX request = celá response v RAM před odesláním klientovi.
Streaming zakázán. Velké responses (export, velký seznam) zbytečně alokují paměť.

2.12 AjaxAntiforgeryResultFilter — reflection hack

```
// Detekce antiforgery selhání přes název třídy – křehká vazba na interní implementaci:
var typeName = result.GetType().Name;
return typeName.Contains("Antiforgery", StringComparison.OrdinalIgnoreCase);
// ASP.NET Core má přesně pro tohle: IAntiforgeryValidationFailedResult
```

2.13 Export — 17 souborů pro Word dokument

```
Services/Export/
├─ IWordExportService.cs
├─ OpenXmlWordExportService.cs
├─ IWordExportHeaderSectionWriter.cs
├─ OpenXmlWordHeaderSectionWriter.cs
├─ IWordExportRecordsSectionWriter.cs
├─ OpenXmlWordRecordsSectionWriter.cs
├─ IWordExportRecordHeaderWriter.cs
├─ OpenXmlWordRecordHeaderWriter.cs
├─ IWordExportRecordCommentsCellWriter.cs
├─ OpenXmlWordRecordCommentsCellWriter.cs
├─ IWordExportRecordDeadlinesCellWriter.cs
├─ OpenXmlWordRecordDeadlinesCellWriter.cs
├─ IWordExportRecordPeopleCellWriter.cs
├─ OpenXmlWordRecordPeopleCellWriter.cs
├─ IWordExportRichHtmlParagraphWriter.cs
├─ OpenXmlWordRichHtmlParagraphWriter.cs
└─ OpenXmlWordElements.cs
```

17 souborů pro vygenerování Word dokumentu. Každá sekce dokumentu má vlastní interface + implementaci. Funkčně ekvivalentní řešení vyžaduje 2 soubory.

2.14 DbContext — 44 DbSetů, vše v jednom souboru

`OnModelCreating` je jedna obrovská metoda bez struktury. Standard je

`IEntityTypeConfiguration<T>`

třídy — každá entita zvlášť, přehledné, udržovatelné.

2.15 site.js — 9 016 řádků monolitu

Jeden soubor, vše pohromadě, žádné moduly, žádný bundling. Prohlížeč parsuje 9K řádků při každém načtení stránky. Nikdo neví co co dělá, nikdo se nebojí smazat "nepotřebný" kód.

2.16 Harmonogram — Gantt výpočty v Razor šabloně

`Views/Projekty/Detail.cshtml` — pro každou položku harmonogramu v `@foreach` loopu:

```
var compactAxisStart = item.DatumZalozeni.Date;
var compactAxisEnd = new[] { item.TerminUkonceni.Date, item.SkutečneDokonceni.Date }.Max();
var compactAxisDays = Math.Max(1, (compactAxisEnd - compactAxisStart).Days);
Func<DateTime, double> toCompactPercent = value =>
    Math.Clamp(((value.Date - compactAxisStart).Days * 100d) / compactAxisDays, 0d, 100d);
Func<double, string> formatPercent = value => value.ToString("0.####", CultureInfo.InvariantCulture);
var compactDeadlinePercent = toCompactPercent(item.TerminUkonceni.Date);
var compactTodayPercent = toCompactPercent(DateTime.Today);
var breakdownDates = item.Kroky.SelectMany(krok => new[] { krok.PlanStart.Date, ... }).ToList();
var breakdownAxisDays = Math.Max(1, (breakdownAxisEnd - breakdownAxisStart).Days);
Func<DateTime, double> toBreakdownPercent = value => ...;
```

Lambda funkce, LINQ, `Math.Max`, `Math.Clamp`, `DateTime.Today` — vše v Razor šabloně. Tyto výpočty patří do `HarmonogramUkolViewModel` jako pre-computed properties. Razor šablona dostane hotová čísla (`double CompactDeadlinePercent`, `double BreakdownTodayPercent`) a jen je vyrendruje do HTML atributů.

2.17 Views — rozsah logiky je systémový problém

Jde o systémový vzor, ne o izolované výjimky. Sken přes celé `Views/` :

Metrika	Počet
<code>var</code> deklarace ve Views	357
<code>Url.Action()</code> volání ve Views	44
<code>ViewBag.</code> reference ve Views	16
LINQ (<code>.GroupBy</code> , <code>.OrderBy</code> , <code>.Select</code> , <code>.Where</code>) ve Views	20+

`Views/Nastaveni/_DetailPanel.cshtml` má například:

```
var selectedProjekty = Model.Projekty
    .Where(p => map.ProjektIds.Contains(p.Id))
    .Select(p => p.Nazev)
    .ToList();
```

Views/Jednani/Detail.cshtml má:

```
.GroupBy(x => string.IsNullOrEmpty(x.StavUcasti) ? "Bez stavu" : x.StavUcasti)
.OrderBy(x => x.Key, StringComparer.CurrentCultureIgnoreCase)
.Select(x => $"{x.Key}: {x.Count()}")
```

Views v současném stavu obsahují více C# logiky než HTML šablonovacích konstrukcí.

2.18 PdfTemplate.cshtml — LINQ v PDF šabloně

Views/Export/PdfTemplate.cshtml — šablona pro tisk/PDF export:

```
.GroupBy(x => x.Subsystem)
.OrderBy(x => x.Key, StringComparer.CurrentCultureIgnoreCase)
// ...
.OrderBy(record => CategoryOrder(record.Kategorie))
// ...
.OrderByDescending(date => date)
```

PDF šablona řadí a seskupuje data. To měl udělat `ExportService` před předáním ViewModelu.

2.19 site.css — 5 024 řádků bez struktury

CSS monolit podobný jako `site.js`. Žádné oddíly, žádná organizace.

Projekty/Detail.cshtml

je 623 řádků, jeho CSS je roztroušeno po celých 5K řádcích souboru.

Přidáš nový komponent → přidáš CSS na konec souboru → za rok nikdo neví co co styluje.

2.20 Program.cs — identifikované nekonzistence

```
// 1. ExceptionHandler míří na /Home/Error, ale HomeController nemá Error action:
app.UseExceptionHandler("/Home/Error");
// Views/Shared/Error.cshtml existuje – správně by mělo být /App/Error nebo /Home/
Error s action
```

```
// 2. public partial class Program; – pro integrační testy přes WebApplicationFact
ory.
// V projektu nejsou žádné testy. Třída je tam pro nic.
public partial class Program;

// 3. Dvojitý route mapping:
app.MapControllers();           // pro attribute-routed controllers
app.MapControllerRoute(...);    // pro conventional routing
// Pokud všechny controllery používají conventional routing, MapControllers() je z
bytečné.
// Pokud používají attribute routing, MapControllerRoute() je zbytečné.
// Jedno z toho tam pravděpodobně nepatří.
```

2.21 JS moduly — správné umístění

Rozdělený `site.js` patří do `wwwroot/js/modules/` — to je standard ASP.NET Core (statické soubory = `wwwroot/`). Název `gantt.js` přejmenovat na `schedule.js` aby odpovídal české terminologii aplikace (harmonogram = schedule).

2.22 ProjektyController — načítání celého seznamu projektů pro jeden item

`ProjektyController.EditProjectModal` a `DeleteProjectModal` dělají toto:

```
// EditProjectModal:
var project = _projectsQueries.BuildProjektyList().FirstOrDefault(item => item.Id
== id);

// DeleteProjectModal:
var project = _projectsQueries.BuildProjektyList().FirstOrDefault(item => item.Id
== id);
```

`BuildProjektyList()` načte **všechny** projekty z DB (SELECT * FROM Projekty, ...). Pak se v C# memory hledá jeden konkrétní projekt přes `FirstOrDefault(id)`.

50 projektů v systému? Načte 50, použije 1. Správně: `GetProjectByIdAsync(id)` — 1 DB dotaz s WHERE.

2.23 RecordCardPartial — načítání celého projektu pro jeden záznam

```
// ZaznamyController.RecordCardPartial:
var model = _recordsService.BuildProjektDetail(projektId); // načte CELÝ projekt
```

```
var record = model.Zaznamy.FirstOrDefault(x => x.Id == zaznamId); // pak hledá 1
záznam
```

`BuildProjektDetail` načte vše — všechny záznamy projektu, všechna jednání, týmové role. Pro zobrazení jedné kartičky záznamu po uložení. Správně: `GetRecordByIdAsync(zaznamId)`.

Toto je pattern, který se opakuje napříč celou aplikací: místo cílených dotazů (`WHERE id = X`) se vždy načte celý dataset a filtruje se v C# paměti.

2.24 ZaznamyController — 13 privátních URL/routing helpers

`ZaznamyController.cs` má 547 řádků a ~13 privátních pomocných metod, z nichž většina řeší routing a URL building logic:

```
private string BuildDeleteRecordReturnUrl(...)
private string BuildRestoreReturnUrl(...)
private string BuildMeetingDetailUrl(int meetingId)
private IActionResult RedirectRecordEditorSaveTarget(...)
private IActionResult BuildSaveAjaxSuccessResult(...) // 40 řádků
private IActionResult BuildDeleteRecordAjaxSuccessResult(...)
private IActionResult ResolveCommentRedirect(...)
private static string NormalizePresentation(...)
private static string NormalizeEditorTab(...)
private static string NormalizeProjectTab(...)
private static string NormalizeDeleteTab(...)
private static string NormalizeRecordEditorUiContext(...)
private string? NormalizeLocalReturnUrl(...)
```

Routing logika patří do pomocné statické utility třídy nebo do ViewModelu, ne do Controlleru.

Controller má orchestrovat — ne buildovat URLs pro 6 různých redirect scénářů.

2.25 Mrtvý kód — "gant" branch který nikdy nenastane

```
// ZaznamyController.NormalizeDeleteTab:
if (string.Equals(tab, "harmonogram", StringComparison.OrdinalIgnoreCase)
    || string.Equals(tab, "gant", StringComparison.OrdinalIgnoreCase)) // ← nikdy
    nepřijde
{
    return "harmonogram";
}
```

Aplikace používá "harmonogram" všude — v URL parametrech, JS, views.
Hodnota "gant" (ani "gantt") nikde v aplikaci nevzniká. Tato větev je mrtvý kód.
Správná oprava: smazat `|| string.Equals(tab, "gant", ...)` celé — není potřeba.

2.26 Export — Modules/Export/ dalších 29 souborů

Vedle `Services/Export/` (17 souborů) existuje ještě `Modules/Export/` s dalšími 29 soubory:

```
Modules/Export/  
├─ ExportDataStore.cs + IExportDataStore.cs  
├─ ExportQueries.cs + IExportQueries.cs  
├─ ExportTemplateQueries.cs + IExportTemplateQueries.cs  
├─ ExportTemplateUseCase.cs + IExportTemplateUseCase.cs  
└─ Queries/  
    └─ ExportAttendanceProjectionBuilder.cs + IExportAttendanceProjectionBuilder.  
cs  
    └─ ExportCommentProjectionBuilder.cs + IExportCommentProjectionBuilder.cs  
    └─ ExportMeetingProjectIdQueryHandler.cs + IExportMeetingProjectIdQueryHandle  
r.cs  
    └─ ExportProjektExistsQueryHandler.cs + IExportProjektExistsQueryHandler.cs  
    └─ ExportRecordProjectionBuilder.cs + IExportRecordProjectionBuilder.cs  
    └─ ExportRecordVisibilityEvaluator.cs + IExportRecordVisibilityEvaluator.cs  
    └─ ExportRoleProjectionBuilder.cs + IExportRoleProjectionBuilder.cs  
    └─ ExportTemplateSummaryBuilder.cs + IExportTemplateSummaryBuilder.cs
```

46 souborů celkem pro export Word dokumentu. Správně: 1 soubor `ExportService.cs` s privátními metodami pro každou sekci. Žádné interfaces, žádné DI pro interní sekce.

2.27 Číselníky — 14 položek v UI, většina by měla být v kódu

Ověřeno z kódu (`DictionariesQueriesUseCase.cs`) — přesný seznam všech 14 číselníků:

Číselník	Smysl v DB + UI?	Odůvodnění
organizace	✓ Ano	Reorganizace MO jsou reálné, mění se
organizacni-celky	✓ Ano	Stejný důvod
subsystemy	✓ Ano	Každý projekt má jiné, přibývají
typy-externich-odkazu	✓ Ano	Nové systémy (PMP, PNF, HTL...) přibývají
vyzvy	✓ Ano	Má atribut <code>Rok</code> — každý rok nová výzva, dynamická data
role-projektu	⚠ Možná	Projektové role (PM, vedoucí) — pokud se nemění, patří do seeder config

Číselník	Smysl v DB + UI?	Odůvodnění
role-subsystemu	⚠ Možná	Stejný argument jako role-projektu
stavy-projektu	❌ Ne	Kód je referenčuje string konstantami (např. "PLAN"). Přejmenování v UI = tiché rozbití logiky. Patří do C# enum.
stavy-ukolu	❌ Ne	Má flag <code>IsFinal</code> — logika v kódu závisí na konkrétních hodnotách. Hardcode.
stavy-jednani	❌ Ne	3 hodnoty (Příprava/Probíhá/Uzavřeno), kód s nimi pracuje přímo. Hardcode.
stavy-ucasti	❌ Ne	Stavy docházky — stabilní, kód je referenčuje. Hardcode.
kategorie-zaznamu	❌ Ne	Kategorie jsou součástí business logiky v kódu. Změna v UI bez změny kódu = nekonzistence.
typy-ukolu	❌ Ne	Stejný argument — kód rozlišuje typy. Hardcode.
harmonogram-kroky	❌ Ne	Technická konfigurace kroků s barvami. Patří do <code>appsettings.json</code> , ne do UI.

Klíčový argument pro hardcode: Stavy a typy jsou referenčovány v C# kódu string konstantami nebo switch výrazy. Kód obsahuje konstrukce jako `if (stav == "PLAN")` nebo `case "SPLNENO":` — tyto řetězce jsou natvrdo v C# a nikdo to nemění.

Problém není „co když admin přejmenuje hodnotu“. Problém je, že **system takovému přejmenování nijak nebrání**. UI ho dovoluje, DB ho uloží, aplikace pak tiše přestane rozpoznávat stavy — žádná exception, žádný log, jen záznamy které vypadají aktivní ale chovají se jako neexistující. Debugování takového problému trvá hodiny, protože symptom (záznamy se nezobrazují) nesouvisí vizuálně s příčinou (přejmenovaná hodnota v číselníku před třemi týdny).

C# enum tento problém fyzicky znemožňuje: neplatná hodnota = chyba při kompilaci nebo parsování, ne tichý průchod.

Navíc: Číselníky jsou v hlavní navigaci přístupné všem uživatelům přes `PermissionKeys.CiselnikyPrefix` . Administrační nástroj patří do Nastavení, ne do hlavního menu.

2.28 Konfigurace oprávnění — kritická citlivost

V Nastavení jsou přes UI editovatelné:

- **Katalog akcí** (`projects.edit` , `records.create` , ...) — klíče musí přesně odpovídat konstantám v `PermissionKeys.cs` . Přejmenování v UI → kód hledá starý klíč → všichni ztratí oprávnění. Tiše.
- **Mapování role → akce** — smazání jednoho řádku → celá role ztratí přístup. Okamžitě.

- **Definice rolí** — vytvoření role v UI která neexistuje v kódu → phantom záznam, nic nedělá.

Jde o dva oddělené problémy:

1. **Křehkost při změně.** Za 6 měsíců přijde požadavek na nové oprávnění. Správce otevře Nastavení, protože tam se konfigurace provádí. Upraví klíč akce. Aplikace přestane tento klíč v kódu rozpoznávat. V logu bude `403 Forbidden` bez vysvětlení příčiny.
2. **Uživatelské rozhraní by nemělo nabízet operace, které jsou nebezpečné.** Tlačítko „Upravit“ vedle `projects.edit` existuje a funguje. Jeho existence je chyba v návrhu — správná architektura znemožňuje špatné operace, nestačí je jen nedoporučovat.

Toto není uživatelská data. Je to konfigurace aplikace. Patří do kódu (seed), ne do UI. Jediné co smí běžet přes UI: **přirazení uživatele k roli**.

2.29 Dvojitá navigace — architektonická nekonzistence prezentační vrstvy

Aplikace má dvě souběžné navigační struktury bez koordinace:

1. `_Layout.cshtml` — globální `<nav class="app-nav">` : Projekty / **Jednání** / Osoby / Číselníky / Nastavení
2. `Views/Projekty/Detail.cshtml` — záložková navigace projektu: Záznamy / Harmonogram / **Jednání** / Tým

Obě položky „Jednání“ mají identický text, identickou barvu, identické HTML. Vedou na zcela jiná místa aplikace — globální `/Jednani/Index` vs. detail projektu s tab parametrem — bez jediného vizuálního rozlišení.

Konkrétní problémy, které z toho plynou:

1. **Aktivní stav navigace je lživý.** Když uživatel klikne na záložku „Jednání“ v projektu, zůstane na `ProjektyController` (URL `/Projekty/Detail/5?tab=jednani`). Globální nav proto zvýrazní „Projekty“, ne „Jednání“. Uživatel čte obsah týkající se jednání, ale nav říká „jsi v Projektech“. Highlightování navigace nesouhlasí s obsahem stránky.
2. **Navigační stav není sdílen.** `_Layout.cshtml` renderuje nav bez přístupu k jakémukoli projektu — `ViewData["CurrentProject"]` neexistuje, sdílený navigační ViewModel neexistuje. Obě struktury vznikly nezávisle a jsou tak i renderovány. Jakákoli koordinace (např. „zobraz v sidebaru kontext aktuálního projektu“) vyžaduje zásah do BaseControlleru a kompletní redesign layoutu — není to přidání jedné podmínky.

3. Přidání nové sekce aplikace nemá jasné pravidlo. Patří do globálního nav? Do záložek projektu? Do obou? Kód žádnou odpověď nenabízí — rozhodnutí se dělá ad hoc a výsledek je vždy další nekonzistence.

4. Uživatel na stránce jednání neví, v jakém kontextu se nachází. Aplikace nemá žádný sdílený datový model, který by propojoval globální navigaci s navigací projektu. Obě vrstvy vznikly nezávisle a žádná z nich neposkytuje informaci o kontextu té druhé — to je architektonická příčina, proč uživatel nemůže z navigačního chrome aplikace poznat, zda se dívá na globální nebo projektová data.

2.30 Nekonzistentní prezentace dat — DRY porušení ve vrstvě Views

Každá sekce projektu používá jiný markup pro prezentaci podobně strukturovaných dat:

Sekce	Markup vzor
Záznamy projektu	Rozbalovací list (<code><details></code> / <code><summary></code>)
Jednání projektu	Grid karet (<code><div class="card"></code>)
Tým projektu	Standardní <code><table></code>
Osoby (globální)	Odlišná <code><table></code> s jinou strukturou sloupců
Číselníky	Kombinace tabulek a karet

`Views/Projekty/` neobsahuje žádný sdílený partial pro seznam entit. Každá view definuje vlastní markup od základu. Jde o porušení DRY na úrovni prezentační vrstvy — společné UI patterny (seznam položek, řádek s akcemi, prázdný stav) jsou duplikovány napříč views bez sdílené komponenty. Přidání nového pole nebo akce do jedné view nevede automaticky ke konzistenci v ostatních.

2.31 Modály pro komplexní editaci — problém komponent s pevnou dimenzí

`ZaznamyController.Edit()` a `Create()` implementují dvě prezentační cesty:

- `EditZaznamPage.cshtml` — celá stránka
- `EditZaznamModal.cshtml` — modál

Obě referencují stejný partial `_EditZaznamForm.cshtml`, který obsahuje Quill editor, záložkové sekce a datové komponenty. Quill inicializuje DOM kontejner s absolutním pozicováním toolbaru a floating panelů. V kontextu modálu s `overflow: auto` a omezenou

výškou dochází k překryvu UI prvků mimo viditelnou oblast a ke scrollování uvnitř modálního okna.

Samotná dualita prezentačních cest je problém na úrovni Controlleru: existují pomocné metody `GetEditorViewPath()` a `NormalizeRecordEditorUiContext()`, jejichž jediným účelem je rozlišit tyto dvě cesty. Stránková varianta (`EditZaznamPage.cshtml`) je funkční a existující — modální cesta přidává komplexitu bez přidané hodnoty pro tento konkrétní formulář.

2.32 Chybějící stránkování a filtrování

`ProjektyController.Detail()` a příslušné UseCase metody načítají celou kolekci záznamů projektu bez OFFSET/LIMIT na úrovni SQL dotazu. Vše je materializováno do paměti a odesláno v jednom HTML těle. Neexistuje žádný URL parametr pro stránkování ani filtrování — GET request na detail projektu vždy vrátí 100 % dat.

Důsledky:

1. **Výkon:** Pro projekt se 100+ záznamy to znamená jeden velký SQL SELECT, velký ViewModel v paměti a velké HTML tělo každý request.
2. **Navigace:** Absence stránkovacích URL parametrů znemožňuje sdílení přímého odkazu na konkrétní pozici v listu nebo sdílení výsledku filtru.
3. **Filtrování subsystemů:** `Views/Projekty/Detail.cshtml` renderuje všechny záznamy a přenechává filtrování na klientskou stranu (JS). Při velké datové sadě to znamená zbytečný přenos dat i pro uživatele, který potřebuje vidět jen jeden subsystem.

2.33 Systém rolí — granularita neúměrná rozsahu aplikace

`Views/Nastaveni/_DetailPanel.cshtml` (ověřeno z kódu) ukazuje 4 oddělené vrstvy autorizace:

1. **AuthzRole** — definice rolí (APP_ADMIN, SUPERADMIN, VS FIS a další)
2. **AuthzAkce** — katalog akcí s klíči (desítky záznamů v DB)
3. **AuthzRoleAkce** — mapovací tabulka role → akce s ScopeMode (GLOBAL, ALL, INCLUDE)
4. **Přiřazení uživatel → role** — individuální přiřazení per projekt nebo globálně

`NastaveniController.cs` implementuje samostatné CRUD akce pro každou z těchto vrstev: `RoleModal`, `PermissionModal`, `RolePermissionModal`, `ToggleRole`, `TogglePermission`, `DeleteRolePermission`. Každá vrstva má vlastní ViewModely, service volání a DB tabulky.

Pro aplikaci na správu zápisů z porad (jedno datové centrum, omezená uživatelská základna) tato granularita přímo nevyplývá z funkčních požadavků. Výsledkem je konfigurační komplexita, kde přidání nové role vyžaduje zásah do čtyř oddělených vrstev. Zjednodušení autorizačního modelu je breaking change — dotkne se DB schématu, všech

`HasPermission()` volání a seeder konfigurace. Doporučeno řešit jako samostatný projekt po dokončení technického refaktoru (Prompty 1–14).

ČÁST 3 — IDEÁLNÍ CÍLOVÁ ARCHITEKTURA

3.1 Cílová struktura složek

```
PmTracker.Web/
|
|— Controllers/
|   |— BaseController.cs          ← abstraktní základ, žádná logika
|   |— ProjektyController.cs
|   |— JednaniController.cs
|   |— ZaznamyController.cs
|   |— OsobyController.cs
|   |— CiselnikyController.cs
|   |— NastaveniController.cs
|   |— ExportController.cs
|   |— ProfilController.cs
|   |— AppController.cs          ← KeepAlive, Error
|
|— Models/
|   |— Entities/
|   |   |— PmTrackerEntities.cs ← EF Core entity třídy (OK jak je)
|   |— ViewModels/
|   |   |— BaseViewModel.cs      ← NOVÝ: base s CurrentHttpContext
|   |   |— JednaniViewModels.cs  ← bool Can*, string *Url computed v controlleru
|   |   |— ProjektyViewModels.cs
|   |   |— ... (ostatní OK)
|
|— Services/
|   |— MeetingService.cs         ← vše pro jednání
|   |— ProjectService.cs        ← vše pro projekty
|   |— RecordService.cs         ← vše pro záznamy
|   |— PeopleService.cs         ← vše pro osoby
|   |— DictionaryService.cs     ← číselníky
|   |— SettingsService.cs       ← nastavení a autorizace
|   |— ExportService.cs         ← Word export (1 soubor)
|   |— ActiveDirectoryService.cs ← AD (OK jak je)
|   |— Shared/
```

```

├── PermissionService.cs
├── PersonIdentityMatcher.cs
├── RichTextService.cs
├── TextNormalizer.cs
├── Data/
│   ├── PmTrackerDbContext.cs
│   └── Configuration/                ← IEntityTypeConfiguration třídy
│       ├── ProjectEntityConfig.cs
│       ├── MeetingEntityConfig.cs
│       ├── RecordEntityConfig.cs
│       ├── LookupEntityConfig.cs
│       └── AuthorizationEntityConfig.cs
├── Middleware/
│   └── RequestTracingMiddleware.cs    ← jen X-Trace-Id, nic víc
├── Filters/
│   └── AjaxAntiforgeryFilter.cs      ← zjednodušeno
├── Views/
│   ├── Shared/
│   │   ├── _Layout.cshtml           ← jen HTML, žádná logika
│   │   ├── _PageHeader.cshtml       ← NOVÝ: jednotný partial pro header stránky
│   │   ├── _ModalLayout.cshtml
│   │   └── Error.cshtml
│   ├── Projekty/
│   ├── Jednani/
│   └── ... (ostatní)
└── wwwroot/
    ├── js/
    │   ├── modules/
    │   │   ├── theme.js
    │   │   ├── session.js
    │   │   ├── modals.js
    │   │   ├── filters.js
    │   │   ├── editor.js
    │   │   ├── schedule.js
    │   │   ├── comments.js
    │   │   └── print.js
    │   ├── site.js                  ← jen import + init
    └── css/
        └── site.css

```

3.2 Pravidla architektury (MUSÍ být dodržena navždy)

CONTROLLER:

- Přijme HTTP request
- Ověří oprávnění (`CurrentUserContext.HasPermission`)
- Zavolá JEDNU service metodu
- Sestaví ViewModel (včetně `bool Can*`, `string *Url`)
- Vrátí View nebo JSON
- ŽÁDNÁ business logika
- ŽÁDNÉ DB dotazy přímo

SERVICE:

- Obsahuje veškerou business logiku
- Přijímá a vrací doménové objekty nebo ViewModely
- Volá DbContext přímo (žádné repository wrappery)
- Všechny metody jsou async (`Task<T>`)
- Přijímá CancellationTokens

VIEW:

- POUZE HTML, `@foreach`, `@if (Model.BoolProperty)`, `@Model.StringProperty`
- ŽÁDNÉ výpočty
- ŽÁDNÉ permission checky
- ŽÁDNÝ ViewBag
- ŽÁDNÉ `Url.Action()` volání (URL jsou v ViewModelu)

VIEWMODEL:

- Obsahuje vše co View potřebuje – hotové, vypočítané
- `bool CanEdit`, `bool CanDelete`, `bool IsLocked`
- `string BackUrl`, `string EditUrl`, `string PrintUrl`
- required properties nebo rozumné defaulty
- `ICollection<T>` pro kolekce

NOVÁ STRÁNKA (např. Dashboard):

1. Přidat `DashboardController.cs`
2. Přidat `DashboardService.cs` s async metodami
3. Přidat `DashboardViewModels.cs`
4. Přidat `Views/Dashboard/Index.cshtml`
5. Hotovo. Žádné další soubory.

3.3 Jak přidat novou stránku (Dashboard) — příklad

Po refaktoru přidání nové stránky vypadá takto:

```
// 1. DashboardViewModel.cs
public sealed class DashboardViewModel : BaseViewModel
{
    public required IReadOnlyList<ProjectSummaryViewModel> RecentProjects { get; init; }
```

```

        public required int OpenMeetingsCount { get; init; }
        public required int PendingTasksCount { get; init; }
    }

// 2. DashboardService.cs
public sealed class DashboardService(PmTrackerDbContext db)
{
    public async Task<DashboardViewModel> BuildDashboardAsync(int osobaId, CancellationTok
    en ct)
    {
        var recentProjects = await db.Projekty
            .Where(p => ...)
            .Select(p => new ProjectSummaryViewModel { ... })
            .ToListAsync(ct);

        return new DashboardViewModel
        {
            RecentProjects = recentProjects,
            OpenMeetingsCount = await db.Jednani.CountAsync(j => ..., ct),
            PendingTasksCount = await db.ProjektoveZaznamy.CountAsync(z => ..., c
            t)
        };
    }
}

// 3. DashboardController.cs
public sealed class DashboardController(
    IUserContextResolver userContextResolver,
    DashboardService dashboardService) : BaseController(userContextResolver)
{
    public async Task<IActionResult> Index(CancellationToken ct)
    {
        var model = await dashboardService.BuildDashboardAsync(CurrentUserContext.
        OsobaId, ct);
        model.CurrentUserContext = CurrentUserContext;
        return View(model);
    }
}

```

3 soubory, žádné zbytečné interface wrappery, žádné delegační vrstvy.

ČÁST 4 — UI KONZISTENCE

4.1 Problém

Každá stránka má jinou HTML strukturu pro stejné UI prvky. Přidání nové stránky = vymyšlení dalšího způsobu. Uživatel cítí vizuální nekonzistenci — různé mezery, různé zarovnání, různé umístění "zpět" tlačítka.

3 různé page headery v aplikaci:

```
<!-- Osoby, Ciselniky: -->
<section class="page-header">
  <h1>Nadpis</h1>
  <div class="header-actions">...</div>
</section>

<!-- Projekty Detail: -->
<section class="page-header page-header-compact">
  <div class="project-header-row">
    <a class="btn small ghost">Zpět</a>
    <span class="project-header-divider">|</span>
    <h1 class="project-title-inline">...</h1>
  </div>
</section>

<!-- EditZaznamPage: -->
<section class="record-editor-page-shell">
  <div class="record-editor-page-header">
    <div class="record-editor-page-header-copy">
      <h1>...</h1>
    </div>
  </div>
</section>
```

4.2 Řešení — jednotný `_PageHeader` partial

Jeden partial view pro header všech stránek:

```
<!-- Views/Shared/_PageHeader.cshtml -->
@model PageHeaderViewModel
<section class="page-header">
  @if (Model.BackUrl != null)
  {
    <a class="btn small ghost" href="@Model.BackUrl">@(Model.BackLabel ?? "Zpět")</a>
  }
  <div class="page-header-content">
    <h1>@Model.Title</h1>
    @if (Model.Subtitle != null)
```

```

        {
            <p class="muted">@Model.Subtitle</p>
        }
    </div>
    @if (Model.Actions != null)
    {
        <div class="page-header-actions">
            @Model.Actions
        </div>
    }
</section>

```

Pak ve všech views:

```

@await Html.PartialAsync("_PageHeader", new PageHeaderViewModel
{
    Title = "Osoby",
    Subtitle = "Globální seznam identit"
})

```

Výsledek: každá stránka používá stejnou HTML strukturu pro header. Přidání nové stránky nevyžaduje rozhodování o způsobu implementace hlavičky.

ČÁST 5 — VÝKON: CHECKLIST

Oblast	Problém	Řešení
Async	0 async metod	Vše převést na async Task
DB dotazy	Celé tabulky do paměti	WHERE před ToList
DB dotazy	Over-fetching	Lightweight query metody
DB zápisy	N+1 v loopech	Batch operations
JavaScript	9 016 řádků, 1 soubor	ES moduly
Statické soubory	Žádná minifikace	Bundling pro produkci
AJAX middleware	Response buffering	Odstranit nebo omezit
Session keep-alive	Každých 5 min polling	OK, ale přesunout do session.js

ČÁST 6 — CODEX PROMPTY

Instrukce pro Codex před každým promptem:

Jsi seniorní C# developer. Píšeš produkční kód pro .NET 8 ASP.NET MVC aplikaci.

Dodržuješ: DRY, SRP, async/await všude, žádné God classes, žádné zbytečné interfaces.

Nepřidáváš nic navíc. Nepřejmenováváš bez důvodu. Neměníš DB schéma.

PROMPT 1 — Smazání Modules/ a odstranění delegačních vrstev

Proč: `Modules/` je kompletní duplikát `Services/Data/`. Přidává 6 vrstev delegace bez jediného řádku logiky navíc. Každá třída v `Modules/` je 1-3 řádky kódu, které jen předají volání dál.

You are a senior C# developer refactoring a .NET 8 ASP.NET MVC application called PmTracker.Web.

THE PROBLEM:

The project has two parallel service architectures for the same domains.

``Modules/`` contains nothing but delegation wrappers around ``Services/Data/``.

Example – calling `BuildJednaniDetail` goes through 10+ classes:

Controller → `IMeetingsQueries` → `MeetingsQueries` (1 line) → `IBuildJednaniDetailQueryHandler`
→ `BuildJednaniDetailQueryHandler` (1 line) → `IMeetingsDataStore` → `MeetingsDataStore` (1 line)
→ `IPmTrackerDataStore` → `SqlServerDataStore` → `IMeetingDetailQueriesUseCase`
→ `MeetingDetailQueriesUseCase` (actual work)

WHAT TO DO:

1. Delete the entire ``Modules/`` directory.

2. Update each Controller that injects Modules interfaces. Replace with direct Use Case injections

```
from `Services/Data/`:  
- `JednaniController`:  
    Remove: IMeetingsQueries, IMeetingsCommands  
    Add: IMeetingDetailQueriesUseCase, IMeetingListQueriesUseCase, IMeetingWriteCommandsUseCase  
- `ProjektyController`:  
    Remove: IProjectsQueries, IProjectsCommands  
    Add: IProjectDetailQueriesUseCase, IProjectListQueriesUseCase, IProjectCommandsUseCase,  
        IProjectAssignmentCommandsUseCase  
- Apply the same pattern to all other controllers.
```

3. Update all Controller method calls to use the injected UseCase methods directly.

If a Module method was ``_meetingsQueries.BuildJednaniDetail(id)``, it now becomes ``_meetingDetailQueriesUseCase.BuildJednaniDetail(id)``.

4. Remove ``Modules/PmTrackerModuleServiceCollectionExtensions.cs`` call from ``Program.cs``.

5. Update ``Services/Data/DataStoreServiceCollectionExtensions.cs`` – remove any registrations that were only needed for Modules wiring.

RULES:

- Do NOT change any business logic in UseCase classes
- Do NOT change DB schema or EF Core entities
- Do NOT add any new abstractions – only remove delegation layers
- Do NOT rename UseCase methods unless they collide
- After this change: Controller → UseCase → DB. Maximum 2 hops.
- Compile and verify there are no unused using statements left

PROMPT 2 — Rozpad God class `SqlServerDataStore`

Proč: `SqlServerDataStore` implementuje 6 rozhraní a injektuje 20+ závislostí. Je to fasáda, která jen deleguje na UseCase třídy. Zbytečná vrstva.

You are a senior C# developer refactoring a .NET 8 ASP.NET MVC application called `PmTracker.Web`.

THE PROBLEM:

``Services/Data/SqlServerDataStore.cs`` implements 6 interfaces with 20+ injected dependencies.

It is a facade that only delegates to UseCase classes. It adds zero logic of its own.

Example:

```
public void SaveMeeting(SaveMeetingCommand command, CurrentUserContextViewModel user)
    => _meetingWriteCommandsUseCase.SaveMeeting(command, user); // pure delegation
```

WHAT TO DO:

1. Examine `SqlServerDataStore` and identify every method.
2. Each method delegates to one of the injected UseCase services.
3. Remove `SqlServerDataStore` entirely.
4. Remove the 6 composition interfaces it implements:
 - `IPmTrackerDataStore`
 - `IProjectDetailComposition`
 - `IDictionariesQueriesComposition`
 - `IRecordEditorQueriesComposition`

- IRecordWriteCommandsComposition
 - IDictionariesCommandsComposition
5. Update `DataStoreServiceCollectionExtensions.cs`:
 - Remove SqlServerDataStore registration
 - Ensure all UseCase classes are directly registered (most already are)
 6. Update any remaining code that injected IPmTrackerDataStore – replace with the specific UseCase interface.

RULES:

- Do NOT change UseCase class implementations
- Do NOT change business logic
- Do NOT add a replacement God class
- After this: each UseCase class is registered directly and injected directly

PROMPT 3 — Konverze celé aplikace na async/await

Proč: Celá aplikace je synchronní. Každý DB dotaz blokuje ASP.NET thread. Pod zátěží = thread pool exhaustion = pomalé odpovědi nebo timeouty. Toto je nejvýznamnější výkonnostní problém backendu.

You are a senior C# developer refactoring a .NET 8 ASP.NET MVC application called PmTracker.Web.

THE PROBLEM:

The entire application is synchronous. All database operations block ASP.NET threads:

- .ToList() instead of .ToListAsync()
- .FirstOrDefault() instead of .FirstOrDefaultAsync()
- .First() instead of .FirstAsync()
- .Any() instead of .AnyAsync()
- .Count() instead of .CountAsync()
- .SaveChanges() instead of .SaveChangesAsync()

WHAT TO DO:

1. Add `Cancellation token cancellationToken = default` as the LAST parameter to:
 - All service interface methods
 - All service implementation methods
 - All UseCase interface methods and implementations
2. Rename all service methods to async convention:
 - `BuildJednaniDetail(int id)` → `BuildJednaniDetailAsync(int id, CancellationToken ct)`
 - `SaveMeeting(...)` → `SaveMeetingAsync(..., CancellationToken ct)`
 - Apply to ALL methods in ALL UseCase and Service classes

3. Replace ALL synchronous EF Core calls with async equivalents:

- ``ToList()`` → ``ToListAsync(cancellationToken)``
- ``FirstOrDefault(x => ...)`` → ``FirstOrDefaultAsync(x => ..., cancellationToken)``
- ``First(x => ...)`` → ``FirstAsync(x => ..., cancellationToken)``
- ``Any(x => ...)`` → ``AnyAsync(x => ..., cancellationToken)``
- ``Count(x => ...)`` → ``CountAsync(x => ..., cancellationToken)``
- ``SaveChanges()`` → ``SaveChangesAsync(cancellationToken)``

4. Update all Controller actions to async:

- ``public IActionResult Index()`` → ``public async Task<IActionResult> Index(CancellationToken ct)``
- ``await service.MethodAsync(..., ct)`` – pass `HttpContext.RequestAborted` as `ct`

5. Update ``BaseController.OnActionExecutionAsync`` – already async, ensure `ct` flows through.

6. Update ``BaseController.ExecuteValidatedCommand`` and ``ExecuteCommand``:

- Change ``Action operation`` parameter to ``Func<Task> operation``
- Make the method async: ``protected async Task<IActionResult> ExecuteValidatedCommand(...)``
- Await the operation: ``await operation()``

RULES:

- NEVER use `.Result` or `.Wait()` – defeats the purpose entirely
- NEVER use `async void` – always `async Task` or `async Task<T>`
- Use `ConfigureAwait(false)` in service layer methods (not in controllers)
- The `CancellationToken` must flow through the entire call chain
- Every method that awaits must itself be async

PROMPT 4 — Oprava DB dotazů (over-fetching, N+1)

Proč: Aplikace načítá celé tabulky bez filtru. Jeden request na zobrazení 1 úkolu spustí 6+ DB dotazů a načte tisíce řádků. `SaveAttendance` dělá N DB roundtripů místo 1.

You are a senior C# developer refactoring a .NET 8 ASP.NET MVC application called `PmTracker.Web`.

THE PROBLEM:

PROBLEM A – Loading entire tables:

In `MeetingDetailQueriesUseCase`:

```
var people = dbContext.Osoby.AsNoTracking().ToDictionary(x => x.Id);
```

This loads ALL persons without a WHERE filter.

PROBLEM B – Over-fetching for simple operations:

JednaniController.TaskItemPartial() calls BuildJednaniDetailAsync() which loads everything (all tasks, all attendees, all comments) just to find ONE task by ID.
JednaniController.SaveStatus() calls BuildJednaniDetailAsync() just to get meeting.ProjektId.
JednaniController.AddMeetingParticipantModal() same issue.

PROBLEM C – N+1 DB writes in loops:

SaveAttendance() calls SaveAttendanceAsync() inside a foreach loop.
SaveNotes() same. 20 rows = 20 separate DB roundtrips.

WHAT TO DO:

Fix A – Filter before materializing:

Replace:

```
var people = dbContext.Osoby.AsNoTracking().ToDictionary(x => x.Id);
```

With:

```
var requiredPersonIds = [collect the IDs you actually need from other loaded data];  
var people = await dbContext.Osoby.AsNoTracking()  
    .Where(x => requiredPersonIds.Contains(x.Id))  
    .ToDictionaryAsync(x => x.Id, cancellationToken);
```

Fix B – Add lightweight query methods to IMeetingDetailQueriesUseCase:

```
Task<int?> GetMeetingProjectIdAsync(int meetingId, CancellationToken ct);  
Task<JednaniTaskItemPartialViewModel?> GetSingleTaskAsync(int meetingId, int taskId, CancellationToken ct);
```

Use GetMeetingProjectIdAsync in SaveStatus and AddMeetingParticipantModal.

Use GetSingleTaskAsync in TaskItemPartial.

Fix C – Batch operations:

1. Change SaveAttendance service method signature:

```
Task SaveAttendanceBatchAsync(int meetingId, IEnumerable<SaveAttendanceRowItem> rows, CancellationToken ct)
```

2. In implementation: load existing Ucast records once, apply all changes, call SaveChangesAsync once.

Use AddRange/UpdateRange – one transaction, one roundtrip.

3. Same for SaveNotes – one call, one transaction.

4. Update JednaniController to pass the full collection in one call.

RULES:

- Do NOT change HTTP request/response contract (same form fields, same redirects)
- Do NOT change EF Core entity schema
- Always apply .Where() filters BEFORE .ToList()/.ToDictionary()
- One SaveChangesAsync per logical operation, not per item

- New lightweight query methods should use Select projections – load only needed columns

PROMPT 5 — Přesun logiky z Views a Layoutu do ViewModels

Proč: Views a Layout.cshtml obsahují business logiku — permission checky, výpočty, přetypování ViewBag. MVC princip: View je hloupá šablona. Controller/ViewModel dělají myšlení.

You are a senior C# developer refactoring a .NET 8 ASP.NET MVC application called PmTracker.Web.

THE PROBLEM:

Views contain permission checks, computed variables, and ViewBag casts.

_Layout.cshtml has navigation permission checks inline.

This violates the MVC pattern – Views should be dumb HTML templates.

WHAT TO DO:

STEP 1 – Create BaseViewModel:

Create `Models/ViewModels/BaseViewModel.cs`:

```
public abstract class BaseViewModel
{
    public required CurrentUserContextViewModel CurrentUserContext { get; init; }

    public bool CanViewPeopleTab => CurrentUserContext.HasPermissionPrefix(PermissionKeys.PeoplePrefix);
    public bool CanViewCiselnikyTab => CurrentUserContext.HasPermissionPrefix(PermissionKeys.CiselnikyPrefix);
    public bool CanViewSettingsTab => CurrentUserContext.HasPermissionPrefix(PermissionKeys.SettingsPrefix);
}
```

STEP 2 – All ViewModels inherit BaseViewModel:

Update all ViewModel classes to inherit BaseViewModel.

Remove `required` from the inherited properties – set them via base.

STEP 3 – Controllers set CurrentUserContext on the model:

In every Controller action that returns a View:

```
model.CurrentUserContext = CurrentUserContext;
```

STEP 4 – Remove all ViewBag user assignments from BaseController.OnActionExecution Async:

Remove these lines:

```
ViewBag.CurrentUserContext = CurrentUserContext;
```

```

ViewBag.CurrentUserName = CurrentUserContext.DisplayName;
ViewBag.CurrentUserEmail = CurrentUserContext.Email;
ViewBag.CurrentUserOrg = CurrentUserContext.OrganizacniCeleg;
ViewBag.CurrentUserOrgCode = ...;
ViewBag.CurrentUserRoles = CurrentUserContext.RoleKody;
ViewBag.IsGlobalAdmin = CurrentUserContext.IsSuperAdmin;

```

STEP 5 – Update _Layout.cshtml:

Replace ViewBag casts and inline logic:

```

var userContext = ViewBag.CurrentUserContext as CurrentUserContextViewModel;
var canViewPeopleTab = ...;

```

With Model-based access. Note: Layout doesn't have a typed model. Pass the nav permissions

via a separate ViewData key set by BaseController:

```

ViewData["NavPermissions"] = new NavPermissionsViewModel
{
    CanViewPeople = CurrentUserContext.HasPermissionPrefix(PermissionKeys.People
Prefix),
    CanViewCiselniky = ...,
    CanViewSettings = ...
};

```

Layout uses: var nav = ViewData["NavPermissions"] as NavPermissionsViewModel;

STEP 6 – Move all @{ var x = ...; } blocks from Views to ViewModels:

For every view, find @{ ... } blocks and move each computed value to a ViewModel property.

Example from Views/Projekty/Detail.cshtml:

Move: canCreateMeetings, canDeleteMeetings, canManageTeam, canManageRecords, canManageSchedules

```

→ bool CanCreateMeetings { get; init; }
→ bool CanDeleteMeetings { get; init; }
etc. – computed in ProjektyController.Detail() using CurrentUserContext.HasPermission(...)

```

Move: createRecordEditorUrl, returnToProjectUrl, projectPrintUrl, projectWordUrl

```

→ string CreateRecordEditorUrl { get; init; }
→ string ReturnToProjectUrl { get; init; }
etc. – computed in Controller using Url.Action(...)

```

Apply to ALL views. Special cases:

HARMONOGRAM in Views/Projekty/Detail.cshtml:

The schedule tab contains Gantt bar calculations per item:

```

Func<DateTime, double> toCompactPercent = ...
Func<DateTime, double> toBreakdownPercent = ...
var compactAxisDays = Math.Max(...)
var breakdownAxisDays = Math.Max(...)

```

Move ALL of these to HarmonogramUkolViewModel as pre-computed properties:

```

double CompactDeadlinePercent { get; init; }

```

```
double CompactTodayPercent { get; init; }
double BreakdownTodayPercent { get; init; }
string FormatPercent(double v) → just store the formatted string in the ViewModel
```

The View then only does: `style="left: @Model.CompactTodayPercent%"`

PDF TEMPLATE (Views/Export/PdfTemplate.cshtml):

Move all `.GroupBy()`, `.OrderBy()`, `.Select()` calls to `ExportService` before building the `ViewModel`.

The template receives pre-sorted, pre-grouped data.

RULES:

- Views must contain ONLY: HTML tags, `@foreach`, `@if (Model.BoolProperty)`, `@Model.StringProperty`
- No `PermissionKeys` references in views
- No `Url.Action()` calls in views
- No string comparisons in views
- No `ViewBag` casts in views
- No `Func<>`, `Math.*`, `DateTime.Today` in views
- No LINQ (`.GroupBy`, `.OrderBy`, `.Select`, `.Where`) in views
- `ViewModel` properties are computed once in the `Controller/Service`, not re-evaluated per view render

PROMPT 6 — Sjednocení UI — konzistentní page header partial

Proč: Každá stránka v aplikaci má jiný HTML layout pro page header. 3 různé způsoby pro totéž. Vizuálně nekonzistentní, obtížné přidat novou stránku konzistentně.

You are a senior C# developer refactoring a .NET 8 ASP.NET MVC application called `PmTracker.Web`.

THE PROBLEM:

Every page uses different HTML structure for its header. There are at least 3 different patterns:

- `<section class="page-header">` with `<h1>` and `<div class="header-actions">`
- `<section class="page-header page-header-compact">` with `<div class="project-header-row">`
- `<section class="record-editor-page-shell">` with completely custom structure

This makes the UI visually inconsistent and makes adding new pages painful.

WHAT TO DO:

1. Create ``Models/ViewModels/PageHeaderViewModel.cs``:

```
public sealed class PageHeaderViewModel
{
```

```

    public required string Title { get; init; }
    public string? Subtitle { get; init; }
    public string? BackUrl { get; init; }
    public string? BackLabel { get; init; }
    public string? Badge { get; init; }          // e.g. project status
}

```

2. Create `Views/Shared/\_PageHeader.cshtml`:

```

@model PageHeaderViewModel
<section class="page-header">
    @if (Model.BackUrl != null)
    {
        <a class="btn small ghost" href="@Model.BackUrl">@(Model.BackLabel ??
"Zpět")</a>
    }
    <div class="page-header-content">
        <h1>@Model.Title</h1>
        @if (Model.Subtitle != null) { <p class="muted">@Model.Subtitle</p> }
        @if (Model.Badge != null) { <span class="badge">@Model.Badge</span> }
    </div>
</section>

```

(Actions are rendered below the header via @RenderSection or just in the view body)

3. Update ALL views to use this partial instead of their custom header HTML:

```

@await Html.PartialAsync("_PageHeader", new PageHeaderViewModel
{
    Title = Model.PageTitle,
    BackUrl = Model.BackUrl,
    BackLabel = "Zpět na přehled"
})

```

4. Add PageTitle, BackUrl, BackLabel to all ViewModels that need them.
Set them in Controllers.

5. Update site.css:

- Unify .page-header CSS to one definition
- Remove .page-header-compact, .project-header-row, .record-editor-page-shell, .record-editor-page-header custom classes (or keep only if genuinely needed for layout difference)
- Ensure all pages look visually consistent

RULES:

- Do NOT change page functionality
- Do NOT change navigation or tab structure within pages
- Every page must use _PageHeader partial – no exceptions
- CSS for page header must be one unified definition

PROMPT 7 — Oprava BaseController

Proč: BaseController používá Service Locator anti-pattern, metoda

`ExecuteValidatedCommand` má 8 parametrů se 4 lambda funkcemi — nečitelné.

You are a senior C# developer refactoring a .NET 8 ASP.NET MVC application called PmTracker.Web.

THE PROBLEM:

PROBLEM A – Service Locator anti-pattern:

BaseController resolves dependencies at runtime from the DI container:

```
var timeProvider = HttpContext.RequestServices.GetService<TimeProvider>();
var loggerFactory = HttpContext.RequestServices.GetService<ILoggerFactory>();
```

Dependencies must be injected via constructor.

PROBLEM B – ExecuteValidatedCommand has 8 parameters (4 lambdas):

```
protected IActionResult ExecuteValidatedCommand(
    Func<bool> hasPermission,
    string invalidAjaxMessage,
    string invalidFallbackMessage,
    Func<IActionResult> onInvalidRedirect,
    Func<IActionResult> onSuccessRedirect,
    Func<IActionResult>? onAjaxSuccess,
    Action operation,
    Func<Exception, IActionResult>? onExceptionRedirect = null)
```

WHAT TO DO:

Fix A:

Add constructor parameters to BaseController:

```
protected BaseController(IUserContextResolver userContextResolver, TimeProvider
timeProvider, ILogger logger)
```

Update all derived controllers to pass these through to base().

Remove all `HttpContext.RequestServices.GetService<>` calls.

Fix B:

After converting to async (Prompt 3), the method signature becomes:

```
protected async Task<IActionResult> ExecuteCommandAsync(
    Func<bool> hasPermission,
    Func<Task> operation,
    Func<Task<IActionResult>> onSuccess,
    string forbiddenMessage = "Nemáte oprávnění.")
```

Model validation check is done before calling this method in the controller action itself.

This reduces the parameter count from 8 to 4.

Update all callers in all controllers accordingly.

Fix C – Dead code in ZaznamyController:

In ZaznamyController.NormalizeDeleteTab(), there is a dead branch:

```
|| string.Equals(tab, "gant", StringComparison.OrdinalIgnoreCase)
```

The value "gant" (or "gantt") is never passed anywhere in the application.

The app uses "harmonogram" consistently. Delete this branch:

```
Remove: || string.Equals(tab, "gant", StringComparison.OrdinalIgnoreCase)
```

RULES:

- Do NOT change authorization behavior
- Do NOT change error response format (ModalSubmitResultViewModel stays same)
- Do NOT change HTTP contract
- All controllers must compile after this change

PROMPT 8 — Sloučení export services

Proč: Word export má 17 souborů — každá "sekce" dokumentu má vlastní interface + implementaci. Funkčně ekvivalentní implementace vyžaduje 2 soubory.

You are a senior C# developer refactoring a .NET 8 ASP.NET MVC application called PmTracker.Web.

THE PROBLEM:

Services/Export/ has 17 files for generating a Word document.

Each section of the document has its own interface + implementation class.

This is massive over-engineering for a simple document generator.

Files to consolidate (delete these 14 files):

- IWordExportHeaderSectionWriter.cs + OpenXmlWordHeaderSectionWriter.cs
- IWordExportRecordsSectionWriter.cs + OpenXmlWordRecordsSectionWriter.cs
- IWordExportRecordHeaderWriter.cs + OpenXmlWordRecordHeaderWriter.cs
- IWordExportRecordCommentsCellWriter.cs + OpenXmlWordRecordCommentsCellWriter.cs
- IWordExportRecordDeadlinesCellWriter.cs + OpenXmlWordRecordDeadlinesCellWriter.cs
- IWordExportRecordPeopleCellWriter.cs + OpenXmlWordRecordPeopleCellWriter.cs
- IWordExportRichHtmlParagraphWriter.cs + OpenXmlWordRichHtmlParagraphWriter.cs

KEEP (rename if needed):

- IWordExportService.cs – keep the public interface as-is
- OpenXmlWordExportService.cs – merge all logic here
- OpenXmlWordElements.cs – keep as static helper class

WHAT TO DO:

1. Merge all section writer logic into OpenXmlWordExportService as private methods:

```
private void AppendHeader(Body body, PdfExportTemplateViewModel model, string t
```

```

itle) { ... }
    private void AppendRecordsSection(Body body, MainDocumentPart mainPart, ...) {
... }
    private void AppendRecord(Body body, ...) { ... }
    private void AppendComments(Body body, ...) { ... }
    private void AppendDeadlines(Body body, ...) { ... }
    private void AppendPeople(Body body, ...) { ... }
    private OpenXmlElement BuildRichTextParagraph(string html) { ... }

```

2. Delete the 14 section writer files.

3. Update DI registration to just:

```

services.AddScoped<IWordExportService, OpenXmlWordExportService>();
(Remove all section writer registrations)

```

RULES:

- Do NOT change the exported Word document output – same format, same content
- Do NOT change IWordExportService public interface
- Do NOT change ExportController
- Private methods replace the former injected sectionwriters
- Merge only – no logic changes

PROMPT 9 — Oprava Filters a Middleware

Proč: Filter používá reflection hack. Middleware bufferuje celé responses v paměti zbytečně.

You are a senior C# developer refactoring a .NET 8 ASP.NET MVC application called PmTracker.Web.

THE PROBLEM:

PROBLEM A – Reflection hack in AjaxAntiforgeryResultFilter:

```

var typeName = result.GetType().Name;
return typeName.Contains("Antiforgery", StringComparison.OrdinalIgnoreCase);

```

ASP.NET Core provides IAntiforgeryValidationFailedResult for exactly this purpose.

PROBLEM B – AjaxResponseContractGuardMiddleware buffers ALL AJAX responses in memory:

```

await using var responseBuffer = new MemoryStream();
context.Response.Body = responseBuffer;
await next(context);
// then validates JSON

```

This prevents streaming and wastes memory for every AJAX request.

WHAT TO DO:

Fix A:

Replace the type-name string check with interface check:

```
if (result is not IAntiforgeryValidationFailedResult) return next();
```

Fix B:

Evaluate what `AjaxResponseContractGuardMiddleware` actually catches in practice.

If it catches cases where the server accidentally returns HTML instead of JSON for AJAX:

- This should be caught at development time by proper code review, not at runtime
- Remove `AjaxResponseContractGuardMiddleware` entirely
- Add a development-only check using `IHostEnvironment.IsDevelopment()` if needed
- Remove it from `Program.cs`

If it also handles session-expiry redirects (server returns 302 instead of JSON):

- Keep ONLY that specific check using `context.Response.OnStarting()` callback
- No full response buffering

Update `Program.cs` accordingly.

RULES:

- Do NOT weaken CSRF protection
- Do NOT change error response JSON format
- Do NOT remove `AjaxErrorCodes` constants
- Preserve all logging

PROMPT 10 — Rozdělení `site.js` na ES moduly

Proč: 9 016 řádků v jednom souboru. Parsuje se vše při každém načtení. Nikdo neví co co dělá. Každá změna riskuje rozbití jiné části.

You are a senior JavaScript developer refactoring a .NET 8 ASP.NET MVC application called `PmTracker.Web`.

THE PROBLEM:

``wwwroot/js/site.js`` is a 9,016-line monolithic IIFE containing all client-side logic.

It mixes concerns: theming, session management, modals, filters, Gantt charts, rich text editor, comments, print preferences – all sharing one closure scope.

WHAT TO DO:

Create ``wwwroot/js/modules/`` directory and split into these files:

1. `modules/theme.js`

Responsible for: dark/light/auto mode, localStorage persistence, system preference detection

Storage key: "pmtracker.theme.mode"
State: mediaDark, themeStorageKey
Export: export function initTheme() { ... }

2. modules/session.js

Responsible for: keep-alive polling (/App/KeepAlive every 5min), session stale detection

Constants: keepAliveEndpointPath, keepAliveIntervalMs (5min), keepAliveTimeoutMs (10s),
keepAliveFailureThreshold (2), sessionStaleErrorCode, sessionExpiredErrorCode

State: sessionState object
Export: export function initSession() { ... }

3. modules/modals.js

Responsible for: modal open/close, focus trap, accessibility, AJAX modal loading

Constants: modalFocusableSelector
State: modalState
Export: export function initModals() { ... }

4. modules/filters.js

Responsible for: project filter UI, chip row, fuzzy search (scoreSearchCandidate),

filter persistence to localStorage

Constants: projectFilterConfigs, projectFilterStoragePrefix, legacyProjectFilterPrefixes

State: filter state per project
Export: export function initFilters() { ... }

5. modules/editor.js

Responsible for: Quill rich text editor, record editor chooser, draft persistence (12h TTL)

Constants: recordEditorDraftStoragePrefix, recordEditorDraftTtlMs, recordEditorPreferenceStorageKey

State: recordEditorState
Export: export function initEditor() { ... }

6. modules/schedule.js

Responsible for: harmonogram tab expand/collapse ("Rozpad" button), floating panel registry,

rainbow canvas for segment text width measurement,
schedule item pinning/expanding state persistence

State: rainbowMeasureCanvas, globalFloatingRoot, floatingPanelRegistry
Export: export function initSchedule() { ... }

NOTE: The Gantt BAR percentages (toCompactPercent, toBreakdownPercent etc.) are NOT here –

those are moved to the ViewModel in Prompt 5. This module handles only UI interactions.

7. modules/comments.js

Responsible for: comment sort direction persistence

Storage key: "pmtracker.comments.sortDirection"

Export: export function initComments() { ... }

8. modules/print.js

Responsible for: print format preferences, print chooser popover hover behavior

Storage key: "pmtracker.print.preferredFormat"

State: printState

Export: export function initPrint() { ... }

New site.js (minimal bootstrap only):

```
import { initTheme } from './modules/theme.js';
import { initSession } from './modules/session.js';
import { initModals } from './modules/modals.js';
import { initFilters } from './modules/filters.js';
import { initEditor } from './modules/editor.js';
import { initSchedule } from './modules/schedule.js';
import { initComments } from './modules/comments.js';
import { initPrint } from './modules/print.js';

document.addEventListener('DOMContentLoaded', () => {
  initTheme();
  initSession();
  initModals();
  initFilters();
  initEditor();
  initSchedule();
  initComments();
  initPrint();
});
```

Update Views/Shared/_Layout.cshtml:

Change: <script src="~/js/site.js"></script>

To: <script src="~/js/site.js" type="module"></script>

RULES:

- Each module uses module-level variables for state – NO global window.* variables
- Use ES module syntax (import/export) – no IIFE, no require()
- Preserve ALL existing functionality – do not change behavior
- Remove any dead code or listeners you find during the split
- If two modules need to communicate, use a small shared events.js with CustomEvent
- Add JSDoc comment to each exported function
- Keep legacy storage key cleanup (legacyProjectFilterPrefixes etc.) in filters.js init

PROMPT 11 — Sloučení zbývajících Services/Records + Services/Data UseCase

Proč: Po smazání `Modules/` stále existují redundantní UseCase vrstvy v `Services/Data/` pro domény, kde by stačil jednoduchý service soubor.

You are a senior C# developer refactoring a .NET 8 ASP.NET MVC application called PmTracker.Web.

CONTEXT:

After previous refactoring steps, `Modules/` is gone and `SqlServerDataStore` is gone. We now have UseCase classes in `Services/Data/` injected directly into controllers. But there are still many thin UseCase classes that just wrap DB queries.

WHAT TO DO:

Consolidate by domain into focused service classes:

1. Create ``Services/MeetingService.cs``:

Merge: `MeetingDetailQueriesUseCase`, `MeetingListQueriesUseCase`, `MeetingWriteCommandsUseCase`,

`MeetingStatePolicy` (keep as static helper within)

Interface: `IMeetingService` with all methods

2. Create ``Services/ProjectService.cs``:

Merge: `ProjectDetailQueriesUseCase`, `ProjectListQueriesUseCase`, `ProjectCommandsUseCase`,

`ProjectAssignmentCommandsUseCase`, `RecordNumberAllocator` (move here or keep separate)

Interface: `IProjectService`

3. Create ``Services/RecordService.cs``:

Merge: `RecordEditorQueriesUseCase`, `RecordWriteCommandsUseCase`, `RecordCommentCommandsUseCase`,

`RecordValidationContracts` (as private helper), `ScheduleTimelineCalculator` (as private helper)

Interface: `IRecordService`

4. Create ``Services/PeopleService.cs``:

Merge: `PeoplePageQueriesUseCase`, `PersonCommandsUseCase`, existing `PeopleService/PeopleDataStore`

Interface: `IPeopleService`

5. Create ``Services/DictionaryService.cs``:

Merge: `DictionariesQueriesUseCase`, `DictionariesCommandsUseCase`, `DictionariesService`,

`DictionarySecurityPolicy`

Interface: `IDictionaryService`

6. Update all controllers to inject the new consolidated interfaces.
7. Remove the old UseCase files and their interfaces.
8. Update DataStoreServiceCollectionExtensions.cs with new registrations.

TARGET: Services/ should have ~8 files. Not 107.

RULES:

- Same async/await rules as Prompt 3
- Do NOT duplicate method implementations – merge, don't copy
- Keep Services/Common/ for shared utilities (PermissionEvaluationService, PersonIdentityMatcher, RichTextContentService, TextNormalizer, CommentAuthorizationPolicy)
- Keep Services/ActiveDirectory/ as-is – it's fine
- Do NOT change DB queries or business logic
- After this: Controller injects 1-2 services, service injects DbContext + shared utilities

PROMPT 12 — Oprava Program.cs

Proč: Exception handler míří na neexistující endpoint, zbytečný `partial class Program` a pravděpodobně dvojí route mapping.

You are a senior C# developer refactoring a .NET 8 ASP.NET MVC application called PmTracker.Web.

THE PROBLEM:

PROBLEM A – ExceptionHandler points to wrong path:

```
app.UseExceptionHandler("/Home/Error");
```

HomeController may not have an Error action. Views/Shared/Error.cshtml exists.
The correct endpoint should match an actual controller action.

PROBLEM B – public partial class Program is unused:

```
public partial class Program; // at the bottom of Program.cs
```

This exists for WebApplicationFactory in integration tests. There are NO tests in this project.

Remove it unless tests are planned immediately.

PROBLEM C – Double route mapping:

```
app.MapControllers();           // attribute routing
app.MapControllerRoute(...);    // conventional routing
```

Check which routing style the controllers actually use. Remove the one that is unused.

WHAT TO DO:

1. Check HomeController – if it has no Error action, change UseExceptionHandler to

```
"/App/Error"
```

```
and add an Error action to ApplicationController that returns View("~/Views/Shared/Error.cshtml").
```

2. Remove `public partial class Program;` unless tests exist.

3. Check all controllers – if none use `[Route]` or `[HttpGet("path")]` attribute routing,

```
remove `app.MapControllers()`. Keep only `app.MapControllerRoute(...)`.
```

RULES:

- Do NOT change the default route pattern
- Do NOT remove error handling – fix the endpoint target
- Do NOT change middleware order

PROMPT 13 — DbContext konfigurace rozdělení

Proč: `OnModelCreating` je jedna obrovská metoda s konfigurací 44+ entit. Nepřehledné.

You are a senior C# developer refactoring a .NET 8 ASP.NET MVC application called `PmTracker.Web`.

THE PROBLEM:

`PmTrackerDbContext.OnModelCreating()` configures 44+ entities in one giant method.

WHAT TO DO:

1. Create directory `Data/Configuration/`

2. Create entity configuration classes using `IEntityTypeConfiguration<T>`:

- `ProjectEntityConfiguration.cs`` – `ProjektEntity`, `ObsazeniProjektuEntity`, `ProjektSubsystemEntity`,
`ObsazeniSubsystemuProjektuEntity`
- `MeetingEntityConfiguration.cs`` – `JednaniEntity`, `UcastEntity`, `VyjadreniEntity`
- `RecordEntityConfiguration.cs`` – `ProjektovyZaznamEntity`, `ZaznamHarmonogramHodnotaEntity`,
all `ZaznamHistorie*` entities, `ZaznamExterniOdkazEntity`, `ZaznamSpolupraceEntity`
- `LookupEntityConfiguration.cs`` – all `CiselnikStavu*`, `CiselnikKategorie*`, `CiselnikTypu*`,
`CiselnikRoli*`, `CiselnikOrganizace*`, `SubsystemEntity`, `HarmonogramSablonaEntity`, `HarmonogramTypEntity`
- `AuthorizationEntityConfiguration.cs`` – all `Authz*` entities
- `PersonEntityConfiguration.cs`` – `OsobaEntity`

3. Replace `OnModelCreating` content with:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
```

```
{
    modelBuilder.ApplyConfigurationsFromAssembly(typeof(PmTrackerDbContext).Assembly);
}
```

RULES:

- Do NOT change any table names, column names, or foreign key configurations
- Do NOT add or remove any entity mappings
- The resulting DB schema must be 100% identical
- Do NOT create migrations – only reorganize configuration code

PROMPT 15 — Zjednodušení číselníků

Proč: 14 číselníků v DB (ověřeno z `DictionariesQueriesUseCase.cs`). Většina z nich jsou stavy a typy referenčované v C# string konstantami — načítají se z DB dynamicky, ale kód s nimi pracuje jako se statickými hodnotami. UI editace těchto hodnot = tichá rozbití logiky bez chyby. Statické hodnoty patří do C# enums. Zbývající dynamické číselníky patří do Nastavení, ne do hlavní navigace.

You are a senior C# developer refactoring a .NET 8 ASP.NET MVC application called PmTracker.Web.

THE PROBLEM:

The application has 14 lookup tables (číselníky) managed through a dedicated UI section accessible to all users in the main navigation. Most of these are static values that never change and should not be editable at runtime.

WHAT TO DO:

STEP 1 – Hardcode static enumerations in C#:

Create ``Models/Enums/PmTrackerEnums.cs`` with:

```
public enum StavProjektu { Plan, Aktivni, Pozastaveno, Ukonceno, Archivovano }
public enum StavUkolu { Rozpracovano, CekaSe, Pozastaveno, Splneno }
public enum StavJednani { Priprava, Probiha, Uzavreno }
public enum TypUkolu { Ukol, Pozadavek, Riziko, Problem } // verify exact values from DB
```

Create ``Models/Enums/EnumDisplayNames.cs`` with Czech display name mappings:

```
public static class StavProjektuNames
{
    public static string Get(StavProjektu stav) => stav switch
    {
```

```

        StavProjektu.Plan => "Plán",
        StavProjektu.Aktivni => "Aktivní",
        // ...
    };
}

```

STEP 2 – Replace DB lookups with enum references:

Anywhere the code loads `stavyProjektu`, `stavyUkolu`, `stavyJednani`, `typyUkolu` from DB:

```

Replace: dbContext.CiselnikStavuProjektu.ToList()
With: Enum.GetValues<StavProjektu>().Select(s => new SelectListItem(s.ToString(), StavProjektuNames.Get(s)))

```

STEP 3 – Remove these číselník DB tables from the UI (keep tables in DB, remove UI management):

Remove UI for: `CiselnikStavuProjektu`, `CiselnikStavuUkolu`, `CiselnikStavuJednani`, `CiselnikTypuUkolu`, `CiselnikRoli` (if duplicated with settings roles)

Keep in DB for migration compatibility but stop loading them dynamically for dropdowns.

STEP 4 – Move Číselníky from main navigation to Nastavení:

In `_Layout.cshtml`: remove the "Číselníky" nav link.

Add a "Číselníky" section inside Nastavení – visible only to users with `CiselnikyEdit` permission.

Show only the genuinely dynamic číselníky: `organizace`, `organizacni-celky`, `subsystemy`, `typy-externich-odkazu`, `vyzvy`.

For `role-projektu` and `role-subsystemu`: evaluate whether values change in practice. If not, include them in the seeder from Step 1 and remove from UI.

RULES:

- Do NOT change DB schema – keep all tables, only stop using them dynamically
- Do NOT change existing enum string values stored in DB – map them to C# enum names carefully
- Existing records with old string values must still display correctly
- After this: dropdowns for `stavy/typy` use C# enums, not DB queries
- Číselníky section is admin-only and shows only 4 items

PROMPT 16 — Stránkování a filtrování seznamů

Proč: `ProjektController.Detail()` načítá celou kolekci záznamů projektu bez OFFSET/LIMIT — žádný SQL stránkovací parametr neexistuje. Filtrování je delegováno na klienta (JS). Neexistuje žádný URL parametr pro stránku nebo filtr subsystému — sdílení odkazu na konkrétní místo v listu nebo na filtrovaný pohled není možné. Pro projekt s 80+ záznamy = jeden velký SQL SELECT každý page load.

You are a senior C# developer refactoring a .NET 8 ASP.NET MVC application called PmTracker.Web.

THE PROBLEM:

Record lists are rendered as infinite scroll – all records on one page.

For a project with 80+ records, the user must scroll endlessly.

There is no quick "show only my subsystem" filter.

WHAT TO DO:

STEP 1 – Add server-side pagination to ZaznamyController/ProjectService:

Add pagination parameters to BuildProjektDetail or create a dedicated paged query:

```
Task<PagedResult<ZaznamViewModel>> GetZaznamyPagedAsync(
    int projektId,
    int page,
    int pageSize,
    string? subsystemFilter,
    string? stavFilter,
    CancellationToken ct)
```

PagedResult<T>:

```
public sealed class PagedResult<T>
{
    public required IReadOnlyList<T> Items { get; init; }
    public required int TotalCount { get; init; }
    public required int Page { get; init; }
    public required int PageSize { get; init; }
    public int TotalPages => (int)Math.Ceiling(TotalCount / (double)PageSize);
}
```

Default page size: 15 records.

STEP 2 – Add pagination UI partial:

Create `Views/Shared/\_Pagination.cshtml`:

```
@model PaginationViewModel
<nav class="pagination" aria-label="Stránkování">
    @if (Model.HasPrevious) { <a href="?page=@(Model.Page-1)&...">< Předchozí</a>
}
    <span>Stránka @Model.Page z @Model.TotalPages (@Model.TotalCount záznamů)</span>
    @if (Model.HasNext) { <a href="?page=@(Model.Page+1)&...">Další ></a> }
</nav>
```

STEP 3 – Add quick subsystem filter buttons above the record list:

In the Záznamy tab, above the list, render subsystem filter pills:

[Vše] [Účetnictví] [HR] [IT] ...

One click filters the list to that subsystem (page reload with ?subsystem=XXX).

Active subsystem highlighted. Preserve in URL for shareability.

STEP 4 – Add stav (status) filter dropdown:

Next to subsystem pills, add a dropdown: "Stav: Vše / Rozpracováno / Čeká se / Splněno"

Both filters combine: `subsystem=Účetnictví&stav=Rozpracovano`

STEP 5 – Update Controller:

`ProjektyController.Detail()` reads page, subsystem, stav from query string.

Passes them to the service. Builds `PaginationViewModel` for the partial.

RULES:

- Do NOT change the record card HTML/CSS – only change how many are shown
- Filters must be bookmarkable (in URL query string)
- Default: page=1, all subsystems, all statuses
- When filter changes, reset to page=1
- Do NOT implement infinite scroll or JavaScript pagination – server-side only

PROMPT 17 — Editace záznamu vždy na vlastní stránce, ne v modálu

Proč: `ZaznamyController` implementuje dvě prezentační cesty pro stejný formulář

(`_EditZaznamForm.cshtml`): stránková (`EditZaznamPage.cshtml`) a modální

(`EditZaznamModal.cshtml`). Pomocné metody `GetEditorViewPath()` a

`NormalizeRecordEditorUiContext()` v Controlleru existují výhradně pro toto rozvětvení.

Formulář obsahuje Quill editor, jehož toolbar používá absolutní pozicování — v modálu s

`overflow: auto` dochází k překryvu UI prvků mimo viditelnou oblast. Stránková varianta je plně funkční a existující — modální cesta přidává komplexitu Controlleru bez přidané hodnoty.

You are a senior C# developer refactoring a .NET 8 ASP.NET MVC application called `PmTracker.Web`.

THE PROBLEM:

Record editing (`ZaznamEditViewModel`) can open as a modal or as a full page.

The modal presentation is used in some flows but the form is too large for a modal (rich text editor, schedule tab, external links, history) – users must scroll inside the modal,

date pickers have positioning issues.

WHAT TO DO:

STEP 1 – Force page presentation for record editing:

In `ZaznamyController.Edit()` and `ZaznamyController.Create()`:

Always use `PresentationPage` – remove the modal presentation path.

Delete `GetEditorViewPath()` – always return `"~/Views/Projekty/EditZaznamPage.cshtml"`.

STEP 2 – Remove `EditZaznamModal.cshtml`:

Delete ``Views/Projekty/EditZaznamModal.cshtml`` – no longer used.

Remove all references to modal presentation in `ZaznamyController`.

STEP 3 – Remove presentation parameter from URLs:

Remove `?presentation=modal` from all `Url.Action()` calls that open the record editor.

The `data-record-editor-url` attributes in views currently pass `presentation=modal` – remove this.

Opening the editor now always navigates to the full page.

STEP 4 – Update AJAX success handling:

After saving a record (`ZaznamyController.Save` AJAX response), instead of refreshing a modal:

Return a page redirect URL: `refreshScope = "page"`, `refreshUrl = project detail URL`

This is already handled for `PresentationPage` – ensure it's the only path.

STEP 5 – Fix date picker positioning:

In the record editor form (`Views/Projekty/_EditZaznamForm.cshtml`), ensure date input fields

are rendered with sufficient surrounding space. If a custom date picker component is used

in `site.js`, ensure it appends to `document.body`, not to the nearest positioned ancestor.

RULES:

- Keep modal presentation ONLY for: confirm dialogs, simple single-field modals (new project, new meeting, add team member, delete confirmations)
- Record editing is always a full page – no exceptions
- Do NOT change the record editor form fields or tabs
- Back button on `EditZaznamPage` must return to the correct project tab

PROMPT 18 — Sidebar navigace místo top nav + záložek

⚠ Tento prompt vyžaduje vizuální specifikaci před implementací.

Doporučeno: nejdřív nakreslit wireframe nebo popsat přesnou strukturu sidebaru, pak teprve zadat Codexu. Bez toho udělá další cirkus.

Proč: `_Layout.cshtml:86` obsahuje globální link `Jednání` → `/Jednani/Index`.

`Views/Projekty/Detail.cshtml` obsahuje záložkovou navigaci se samostatnou záložkou `Jednání` → projekt detail s tab filtrem. Dvě nezávislé navigační struktury bez sdíleného

modelu nebo koordinace — entita Jednání je dosažitelná ze dvou různých kontextů bez vizuálního rozlišení. Z kódu: žádný sdílený navigační ViewModel neexistuje, layout a detail view definují své navigační struktury nezávisle.

You are a senior C# developer and UX-aware developer refactoring a .NET 8 ASP.NET MVC application.

THE PROBLEM:

Navigation exists in two places simultaneously:

1. Top bar: Projekty / Osoby / Číselníky / Jednání / Nastavení
2. Project detail tabs: Záznamy / Harmonogram / Jednání / Osoby (tým)

"Jednání" appears in BOTH – global meetings list and project meetings tab. Users do not understand the difference. The dual navigation adds cognitive load.

WHAT TO DO:

STEP 1 – Replace top nav with left sidebar in `_Layout.cshtml`:

The sidebar contains:

```
[App logo]


---


Projekty      ← navigates to /Projekty
Jednání      ← navigates to /Jednani (global list, rename to "Všechna jednání")
Osoby        ← navigates to /Osoby (admin only)


---


[current project name if on project detail]
Záznamy
Harmonogram
Jednání      ← project-specific meetings
Tým


---


Nastavení    ← bottom, admin only
Číselníky    ← bottom, admin only (moved from main nav)
```

STEP 2 – Update `_Layout.cshtml`:

Replace `<nav class="app-nav">` top navigation with `<aside class="app-sidebar">`. The sidebar is always visible. On mobile it collapses (hamburger toggle). Main content area shifts right to accommodate sidebar width (~220px).

STEP 3 – Update `site.css`:

Replace `.app-nav` styles with `.app-sidebar` styles.

`.app-body` layout changes from vertical stack to horizontal flex:

```
.app-sidebar { width: 220px; flex-shrink: 0; }
.app-main { flex: 1; overflow: auto; }
```

STEP 4 – Project context in sidebar:

When user is on a project detail page, show project sub-navigation in the sidebar.

Pass current project name and ID from BaseController via ViewData["CurrentProject"].
Sidebar shows project section only when ViewData["CurrentProject"] is set.

STEP 5 – Rename global "Jednání" to avoid confusion:
Change the global meetings link label from "Jednání" to "Přehled jednání" to distinguish from project-specific meetings tab.

- RULES:
- Mobile: sidebar collapses to hamburger menu
 - Active link highlighted in sidebar
 - Sidebar state (collapsed/expanded) persisted in localStorage
 - Do NOT change any Controller logic or routing
 - Do NOT change page content – only navigation chrome

ČÁST 7 — POŘADÍ IMPLEMENTACE

Spouštěj prompty v tomto pořadí. Každý po předchozím otestuj (spust' aplikaci, projdi hlavní funkce).

#	Prompt	Nezávislý?	Poznámka
1	Smazání Modules/	Ne — základ	Musí být první
2	Rozpad SqlServerDataStore	Ne — po #1	SqlServerDataStore závisel na Modules
3	Async/await	Ano	Největší práce, nejdůležitější
4	DB dotazy (over-fetching, N+1)	Ne — po #3	Dotazy musí být async
5	Views → ViewModels	Ano	Kdykoliv
6	Konzistentní page header	Ano	Kdykoliv, vizuální dopad
7	BaseController oprava	Ano	Kdykoliv
8	Export sloučení	Ano	Kdykoliv
9	Filters + Middleware	Ano	Kdykoliv
10	site.js moduly	Ano	Kdykoliv, nezávislé na backendu
11	Sloučení UseCase → Services	Ne — po #1,#2	Finální service konsolidace
12	Program.cs oprava	Ano	Kdykoliv, low risk
13	DbContext konfigurace	Ano	Kdykoliv, low risk
14	Konfigurace oprávnění z DB do kódu	Ano	Před ostrým provozem — bezpečnostní priorita

PROMPT 14 — Přesun konfigurace oprávnění z DB do kódu

Proč: Permission klíče, definice rolí a mapování role→akce jsou uloženy v DB a editovatelné přes UI. Jakýkoliv správce může smazat mapování, přejmenovat klíč nebo deaktivovat akci → aplikace se rozbije tiše bez chyby. Toto je konfigurace aplikace, ne uživatelská data.

You are a senior C# developer refactoring a .NET 8 ASP.NET MVC application called PmTracker.Web.

THE PROBLEM:

The Settings UI (Nastavení) allows admins to:

1. Create/edit/deactivate permission ACTION KEYS (e.g. "projects.edit", "records.create")
stored in DB table AuthzAkce
2. Create/edit/delete ROLE→ACTION mappings (which role gets which permission)
stored in DB table AuthzRoleAkce
3. Create/edit ROLES stored in DB table AuthzRole

This is application configuration masquerading as user data. The risk:

- Permission keys MUST match constants in PermissionKeys.cs. If an admin renames a key in the UI,
the C# code still checks for the old hardcoded constant → permission silently denied for everyone.
- An admin deactivates a system action → feature breaks with no error shown.
- An admin deletes a role→action mapping → entire role loses access instantly.
- These are not things any user – including superadmin – should configure at runtime.

WHAT TO DO:

STEP 1 – Create a seed configuration class:

Create `Services/Security/PermissionSeedConfiguration.cs` with all roles, actions, and mappings
as C# readonly data:

```
public static class PermissionSeedConfiguration
{
    public static readonly IReadOnlyList<RoleSeedItem> Roles = new[]
    {
        new RoleSeedItem("APP_ADMIN", "Administrátor aplikace", isSystem: true),
        new RoleSeedItem("SUPERADMIN", "Super administrátor", isSystem: true),
        // ... all existing roles
    };

    public static readonly IReadOnlyList<ActionSeedItem> Actions = new[]
    {
        new ActionSeedItem(PermissionKeys.ProjectsCreate, "Vytvořit projekt", ScopeLevel.Global),
```

```

        new ActionSeedItem(PermissionKeys.ProjectsEdit, "Upravit projekt", Scope
Level.Project),
        // ... all existing actions – key MUST match PermissionKeys constants
    };

    public static readonly IReadOnlyList<RoleActionSeedItem> RoleMappings = new
[]
    {
        new RoleActionSeedItem("APP_ADMIN", PermissionKeys.ProjectsCreate, Scope
Mode.Global),
        // ... all existing mappings
    };
}

```

STEP 2 – Create a startup seeder:

Create `Services/Security/PermissionSeeder.cs`:

```

public sealed class PermissionSeeder(PmTrackerDbContext db)
{
    public async Task SeedAsync(CancellationToken ct)
    {
        // Upsert roles – insert if not exists, update name/description if chang
ed
        // Upsert actions – insert if not exists, never allow deactivation of sy
stem actions
        // Upsert role→action mappings from the seed config
        // Do NOT delete anything – only add/update
        await db.SaveChangesAsync(ct);
    }
}

```

STEP 3 – Run seeder on startup in Program.cs:

```

using (var scope = app.Services.CreateScope())
{
    var seeder = scope.ServiceProvider.GetRequiredService<PermissionSeeder>();
    await seeder.SeedAsync(CancellationToken.None);
}

```

STEP 4 – Remove dangerous UI sections from Nastavení:

In Views/Nastaveni/_DetailPanel.cshtml:

- Remove the "role" section (role CRUD) – roles come from seed only
- Remove the "akce" section (action/permission key CRUD) – keys come from PermissionKeys.cs + seed
- Remove the "role-akce" section (role→action mapping) – mappings come from seed only
- KEEP the "uzivatele-role" section – assigning users to roles is the only runtime operation
- KEEP the "efektivni-prava" section – read-only view is useful

In NastaveniController: remove actions for RoleModal, PermissionModal, RolePermiss

ionModal,
ToggleRole, TogglePermission, DeleteRolePermission – these are no longer needed.

STEP 5 – Update Nastavení navigation:

Remove the now-deleted sections from the settings sidebar/tabs.

The settings page should show only: "Uživatelé a role" and "Efektivní práva".

RULES:

- Do NOT change PermissionKeys.cs constants – they are the source of truth for key strings
- Do NOT change the HasPermission() evaluation logic
- Do NOT remove AuthzAkce, AuthzRole, AuthzRoleAkce DB tables – keep the schema, remove the UI
- The seeder must be idempotent – running it multiple times must produce the same result
- System roles and actions (IsSystem=true) must never be deletable even from code
- After this: permission configuration lives in PermissionSeedConfiguration.cs, only user→role assignments are managed at runtime through the UI

ČÁST 8 — CO ZACHOVAT BEZ ZMĚNY

- `Services/ActiveDirectory/` — AD integrace je OK jak je
- `Data/PmTrackerDbContext.cs` DbSets — zachovat vše, jen přesunout OnModelCreating konfiguraci
- Permission systém (`PermissionKeys` , `HasPermission` , `AuthzRole*` entity) — logika je OK
- Antiforgery ochrana — jen opravit implementaci (Prompt 9)
- Word export funkcionalita — jen sloučit soubory, výstup zachovat
- Czech doménové názvy (Jednani, Zaznamy, Osoba) — konzistentní, zachovat
- `Models/Entities/PmTrackerEntities.cs` — entity jsou OK
- Quill editor integrace — zachovat, jen přesunout do editor.js

ČÁST 9 — DEFINICE "HOTOVO"

Refaktor je hotový, když:

- ☐ `Modules/` složka neexistuje
- ☐ `SqlServerDataStore.cs` neexistuje

- ☐ Žádný Controller neinjektuje více než 3 service závislosti
- ☐ Všechny service metody jsou `async Task<T>`
- ☐ Všechny Controller actions jsou `async Task<IActionResult>`
- ☐ Žádné View neobsahuje `@{ var x = ...; }` bloky
- ☐ Žádné View neobsahuje `ViewBag.CurrentUserContext as ...`
- ☐ Žádné View neobsahuje `PermissionKeys.` reference
- ☐ `site.js` má méně než 30 řádků (jen importy a init)
- ☐ `Services/Export/` má 2 soubory (interface + implementace)
- ☐ `Modules/Export/` složka neexistuje
- ☐ `Services/Data/` složka neexistuje (vše sloučeno do `Services/`)
- ☐ Žádný Controller neprovádí `.FirstOrDefault()` na kolekci načtené bez WHERE filtru
- ☐ Typo "gant" opraveno — `NormalizeDeleteTab` kontroluje "gantt" i "harmonogram"
- ☐ Každá stránka používá `_PageHeader` partial
- ☐ Aplikace se načítá bez JS chyb v konzoli
- ☐ Aplikace funguje — přihlášení, zobrazení projektu, zápis z jednání, export Word
- ☐ `PermissionSeedConfiguration.cs` existuje a seeder běží na startu
- ☐ UI Nastavení zobrazuje pouze "Uživatelé a role" a "Efektivní práva" — žádný CRUD rolí/akcí